Cómo resolver el parcial, paso a paso

Siempre te dan un proyecto que ya funciona (el preenunciado) y, el día del parcial, un CSV con una o más columnas nuevas y una consigna con los mismos cinco puntos. Cambia el tema, no la forma. Esta guía sigue el orden en que conviene trabajar, con el código **antes** y **después** (lo nuevo, marcado) y, en cada paso, qué hacer si te piden otra cosa, según los cinco parciales: el **Parcial 3K1 (Cloud Center)** y los simulacros **Fintech**, **Taller de reparaciones**, **Juegos de mesa** y **Tarjetas de crédito**.

Lo que pide siempre la consigna

1. **Leer los dos formatos** del CSV (el viejo y el nuevo) sin duplicar la carga.
2. **La regla nueva** que trae la columna: una validación distinta, un cálculo distinto, o las dos.
3. **Una operación nueva en el colector**: un conteo, un total o un agrupamiento.
4. **El main** usa el CSV del parcial por defecto y muestra todo, delegando.
5. **La entrega**: tests viejos y nuevos en verde, `DECISIONES.md` y un ZIP.

Dónde se trabaja

Rol	Cómo se llama	Qué tocás
Parser	ParserSolicitudes, ParserServidores…	Las cabeceras, el ancho esperado y el único loop de lectura
Dominio	SolicitudCredito, Servidor, Reparacion, Consumo, Juego	`desdeCampos`, constructor, atributo nuevo, validaciones, cálculo; subclase si cambia un cálculo
Resultado	ResultadoProceso, ResultadoParseo	Casi nunca se toca
Colector	CarteraCreditos, Cloud, Taller, Colector, Servicio	La operación nueva
Main	App, Main	La ruta por defecto y lo que se imprime

Tiempo: **5 min** lectura · **25 min** cambios · **10 min** pruebas · **5 min** entrega.

---

0 Antes de escribir código

Dónde: la consigna, la carpeta del proyecto y la terminal.

1. Leé la consigna y marcá: la **cabecera nueva**, los **valores válidos** de la columna nueva, a qué **estados** afecta, las **fórmulas y ejemplos**, la **operación del colector** y qué tiene que mostrar el **main**.
2. Copiá los tests nuevos **fusionando carpetas** y el CSV a `datos/` con el nombre exacto de la consigna.
3. Corré `mvn test` y anotá qué falla. Lo esperable: los viejos pasan y algunos nuevos fallan. Esa es tu línea base.

Terminal

```
# Desde la raíz del proyecto (la carpeta que tiene el pom.xml)
cp -R tests-nuevos/ar src/test/java/          # Cloud Center y Fintech: el paquete es ar
cp -R tests-nuevos/utnfc src/test/java/       # Taller y Tarjetas: el paquete es utnfc
cp datos-parcial.csv datos/datos-parcial.csv
mvn test
```

Trampa: el ZIP del Parcial 3K1 traía `datos_parcial.csv` (guion bajo) y la consigna pedía `datos-parcial.csv` (guion medio). Si el nombre no coincide, el main no lo encuentra. Usá el de la consigna.

Dato: si un test nuevo no compila, mirá primero la línea `package` : tiene que ser igual a la carpeta donde lo copiaste.

1 Parser: aceptar las dos cabeceras

Dónde: la clase Parser, método `procesar(Path)` o `leer(Path)` .

El mismo loop lee los dos formatos: la **primera línea** decide cuántas columnas tiene que tener **cada** fila. Así una fila de 5 bajo una cabecera de 6 es inválida, y al revés también.

Antes

```
private static final String CABECERA = "idCliente,...,estadoSolicitud";

try (BufferedReader lector = Files.newBufferedReader(archivo, StandardCharsets.UTF_8)) {
    if (!CABECERA.equals(lector.readLine()))
        throw new IllegalArgumentException("encabezado incorrecto");
    String linea;
    while ((linea = lector.readLine()) != null) {
        ...
        if (tokens.length != 5)
            throw new IllegalArgumentException("Cantidad de columnas incorrectas");
```

Después · lo nuevo, marcado

```
private static final String CABECERA_5 = "idCliente,...,estadoSolicitud";
private static final String CABECERA_6 = CABECERA_5 + ",nivelMorosidad";

try (BufferedReader lector = Files.newBufferedReader(archivo, StandardCharsets.UTF_8)) {
    String cabecera = lector.readLine();
    int largoCabecera;
    if (CABECERA_5.equals(cabecera)) largoCabecera = 5;
    else if (CABECERA_6.equals(cabecera)) largoCabecera = 6;
    else throw new IllegalArgumentException("encabezado incorrecto");
    String linea;
    while ((linea = lector.readLine()) != null) {      // UN solo loop
        ...
        if (tokens.length != largoCabecera)
            throw new IllegalArgumentException("Cantidad de columnas incorrectas");
```

El orden adentro del loop es siempre el mismo: **1) ancho** → **2) descarte** (BLACK, ESTABLE, CANCELADA, ANULADO) sin validar nada más → **3) validaciones**.

Trampa: hacer `if (formato viejo) { while… } else { while… }`. Eso es "duplicar el proceso de carga" y lo descuentan.

SI TE PIDEN un separador ; (Juegos de mesa, 9 → 10 columnas)

Mismo patrón. El `;` va en las constantes y en el `split`, y el `-1` conserva el campo vacío del final (un juego sin dificultad tiene 10 campos igual).

```
private static final String CABECERA_9 =
"id;nombre;anio;min_jugadores;max_jugadores;duracion;rating;peso;categoria";
private static final String CABECERA_10 = CABECERA_9 + ";dificultad_cooperativa";
...
String[] campos = Arrays.stream(linea.split(";", -1)).map(String::strip).toArray(String[]::new);
```

SI TE PIDEN varias columnas nuevas (Taller: 8 → 11)

Igual, pero podés sacar el ancho de la cabecera en vez de escribir los números.

```
if (!CABECERA_INICIAL.equals(cabecera) && !CABECERA_EVOLUCIONADA.equals(cabecera))
    throw new IllegalArgumentException("encabezado incorrecto");
int ancho = cabecera.split(",").length;   // 8 u 11
```

SI TE PIDEN un **estado nuevo que es válido** (Taller: ENTREGADA)

Sumalo a la lista de estados que el parser acepta. El descarte (CANCELADA) queda donde estaba, después del ancho.

```
if (!List.of("ABIERTA", "LISTA").contains(c[6]))
if (!List.of("ABIERTA", "LISTA", "ENTREGADA").contains(c[6]))
    throw new IllegalArgumentException("estado desconocido: " + c[6]);
```

SI TE PIDEN "después de quitar los espacios externos"

Ya lo hace el `.map(String::strip)` del preenunciado sobre cada campo: no hay que agregar nada.

Cómo verificar: los tests viejos siguen en verde con `mvn test`.

2 Dominio: que *desdeCampos* acepte los dos anchos

Dónde: la clase del dominio, `desdeCampos(String[])`.

El dominio casi siempre tiene el ancho fijo, repetido. Si arreglás solo el parser, todas las filas nuevas mueren con "Se esperan 5 columnas".

Antes

```
public static SolicitudCredito desdeCampos(String[] tokens) {
    if (tokens.length != 5)
        throw new IllegalArgumentException("Se esperan 5 columnas");
    double ingresos = Double.parseDouble(tokens[2]);
```

Después · lo nuevo, marcado

```java
public static SolicitudCredito desdeCampos(String[] tokens) {
    if (tokens.length != 5 && tokens.length != 6)
        throw new IllegalArgumentException("Se esperan 5 o 6 columnas");
    double ingresos = Double.parseDouble(tokens[2]);
    // la columna nueva se lee SOLO si existe; si no, el valor por defecto
    String nivelMorosidad = tokens.length == 6 ? tokens[5] : "NORMAL";
```

Trampa grande: leer `tokens[5]` en una fila de 5 tira `ArrayIndexOutOfBoundsException`. No es `IllegalArgumentException`: el `catch` del parser no la atrapa y se corta **toda** la carga del CSV viejo.

Dato útil: `Integer.parseInt` y `Double.parseDouble` tiran `NumberFormatException`, que **sí** es hija de `IllegalArgumentException`: un número mal escrito (o vacío) deja la fila inválida sin cortar la lectura.

SI TE PIDEN **un valor por defecto que no es un texto (Servidores: ingresos 1000, Tarjetas: 0 días)**

Mismo ternario, con el tipo que corresponda.

```java
int ingresos = c.length == 6 ? Integer.parseInt(c[5]) : 1000;
int dias = c.length == 6 ? Integer.parseInt(c[5]) : 0;
```

Cómo verificar: corré el main con el CSV del parcial: leídas = procesadas + descartadas + inválidas. Con el CSV viejo tienen que salir los mismos números de siempre.

3 La regla nueva, en el dominio

Dónde: la clase del dominio. **No** en el parser: la consigna pide que la construcción y las reglas queden en el dominio.

Cuatro sub-pasos, siempre en este orden:

1. **Atributo nuevo + getter.**
2. **Constructor nuevo** con el parámetro extra. El viejo se queda y **delega** con `this(...)` pasando el valor por defecto: los tests viejos lo usan y tienen que seguir compilando.
3. La validación de combinaciones en un método `private static` aparte, **afuera** de `desdeCampos`.
4. `if (caso nuevo) { reglas nuevas } else { reglas viejas tal cual }`.

Antes · Fintech

```
private final String estado;

public SolicitudCredito(String id, String tipo, double score, String estado) {
    ... validaciones ...
    this.estado = estado;
}

// en desdeCampos:
    if (ingresos < 1000 || ingresos > 15000)
        throw new IllegalArgumentException("Ingresos no aptos");
    if (deudas < 0 || deudas >= ingresos)
        throw new IllegalArgumentException("Deudas no aptas");
    return new SolicitudCredito(tokens[0], tokens[1], score, tokens[4]);
```

Después · lo nuevo, marcado

```
private final String estado;
private final String nivelMorosidad;

public SolicitudCredito(String id, String tipo, double score, String estado) {
    this(id, tipo, score, estado, "NORMAL");          // el viejo delega
}
public SolicitudCredito(String id, String tipo, double score, String estado, String nivelMorosidad)
{
    ... validaciones ...
    this.estado = estado;
    this.nivelMorosidad = nivelMorosidad;
}

// en desdeCampos:
    if (tokens.length == 6) validarMorosidad(tokens[4], nivelMorosidad);
    if (nivelMorosidad.equals("EN_MORA")) {                // regla nueva
        if (ingresos <= 25000 || deudas < 0 || deudas > ingresos * 0.12)
            throw new IllegalArgumentException("EN_MORA: ingresos o deudas no aptos");
    } else {                                               // reglas viejas, sin tocar
    if (ingresos < 1000 || ingresos > 15000)
        throw new IllegalArgumentException("Ingresos no aptos");
    if (deudas < 0 || deudas >= ingresos)
        throw new IllegalArgumentException("Deudas no aptas");
    }
    return new SolicitudCredito(tokens[0], tokens[1], score, tokens[4], nivelMorosidad);
}

private static void validarMorosidad(String estado, String nivel) {   // AFUERA de desdeCampos
    switch (estado) {
        case "RECHAZADO" -> { if (!nivel.equals("EN_MORA"))
            throw new IllegalArgumentException("RECHAZADO solo admite EN_MORA"); }
        case "APROBADO", "PRE_APROBADO" -> { if (!nivel.equals("NORMAL") &&
!nivel.equals("EN_MORA"))
            throw new IllegalArgumentException("Morosidad inválida: " + nivel); }
        default -> throw new IllegalArgumentException("Estado desconocido: " + estado);
    }
}
public String getNivelMorosidad() { return nivelMorosidad; }
```

Trampas: pasarle `tokens[5]` al constructor (se rompe con 5 columnas: pasale la variable); escribir el método nuevo **adentro** de otro (no compila); cambiar la firma del constructor viejo en vez de agregar uno (los tests viejos dejan de compilar).

SI TE PIDEN un rango solo para algunos estados (Cloud Center: ALERTA y CRITICO, 1000 ≤ ingresos < 1900)

Ojo con los límites: `>= 1000` y `< 1900`. Si después del descarte solo quedan esos estados, podés validar en el constructor de la subclase (como en la solución propuesta).

```
boolean enRiesgo = estado.equals("ALERTA") || estado.equals("CRITICO");
if (c.length == 6 && enRiesgo && (ingresos < 1000 || ingresos >= 1900))
    throw new IllegalArgumentException("ingresos fuera de rango: " + ingresos);
```

SI TE PIDEN valores permitidos y un tope (Tarjetas: 0 / 30 / 90 / 180 días, máximo 500 000 netos si está asegurado)

`List.of(...).contains` para los valores y el tope **inclusive**. Un valor vacío ya falla solo en `Integer.parseInt`.

```
if (!List.of(0, 30, 90, 180).contains(diasCobertura))
    throw new IllegalArgumentException("plazo de cobertura inválido: " + diasCobertura);
if (diasCobertura > 0 && importeCentavos - bonificacionCentavos > 500_000)
    throw new IllegalArgumentException("neto supera el tope asegurado");
```

SI TE PIDEN una columna opcional: vacía quiere decir "normal" (Juegos: dificultad cooperativa)

Si no hay valor, no se valida nada: es un juego normal. Si hay, tiene que ser un entero entre 1 y 10.

```
boolean cooperativo = c.length == 10 && !c[9].isEmpty();
if (cooperativo) {
    int dificultad = Integer.parseInt(c[9]);
    if (dificultad < 1 || dificultad > 10)
        throw new IllegalArgumentException("dificultad fuera de rango: " + dificultad);
}
```

SI TE PIDEN varios campos con valores fijos (Taller: garantía SI/NO, repuestos ≥ 0)

Un `switch` que devuelve el valor o tira la excepción; y la regla del número.

```
boolean garantia = switch (c[8]) {
    case "SI" -> true;
    case "NO" -> false;
    default -> throw new IllegalArgumentException("garantía inválida: " + c[8]);
};
double repuestos = Double.parseDouble(c[9]);
if (repuestos < 0) throw new IllegalArgumentException("repuestos negativos");
```

4 Cuando cambia un cálculo: polimorfismo

Dónde: una **subclase nueva** + `desdeCampos`, que elige qué clase crear.

Señales en la consigna: "mediante polimorfismo", "especialización", "calcula un importe diferente", "conserva las reglas comunes y agrega...". La solución típica: una subclase que **redefine** el método del cálculo y reutiliza el del padre con `super`.

Antes · Taller

```
public static Reparacion desdeCampos(String[] c) {
    ...
    return new Reparacion(c[0], c[2], c[6], prioridad, horas, costoBase, garantia, repuestos);
}
```

Después · lo nuevo, marcado

```
public static Reparacion desdeCampos(String[] c) {
    ...
    return switch (c[10]) {                    // desdeCampos decide qué clase crear
        case "NORMAL" -> new Reparacion(c[0], c[2], c[6], prioridad, horas, costoBase, garantia,
repuestos);
        case "EXPRESS" -> new ReparacionExpress(c[0], c[2], c[6], prioridad, horas, costoBase,
garantia, repuestos);
        default -> throw new IllegalArgumentException("modalidad inválida: " + c[10]);
    };
}

class ReparacionExpress extends Reparacion {
    public ReparacionExpress(/* los mismos parámetros */) { super(/* ... */); }
    @Override
    public double importe() {
        return super.importe() * 1.25;          // reutiliza todo el cálculo común
    }
}
```

La lista sigue siendo `List<Reparacion>`: el colector no cambia, y cuando llama a `importe()` Java ejecuta la versión de cada objeto.

Trampas: usar `instanceof` o un `if (tipo == ...)` en el colector o el main (justo lo que el polimorfismo evita); copiar la fórmula entera en la subclase en vez de llamar a `super`.

SI TE PIDEN que el dato nuevo entre en una fórmula que ya existe (Cloud Center: el ingreso por hora en el costo de riesgo)

"Sin alterar la estructura de objetos": la fórmula queda en el padre y usa un método que la subclase redefine. Así `total()`, los filtros y los totales por región toman la regla nueva sin tocarlos.

```
public int costoRiesgoOperativo() {
    return (ingresoPorHora() * indiceCarga) / 100;      // antes: 1000 fijo
}
protected int ingresoPorHora() { return 1000; }         // el formato viejo

class ServidorConIngresos extends Servidor {
    private final int ingresos;
    ...
    @Override protected int ingresoPorHora() { return ingresos; }
}
```

SI TE PIDEN sumar algo al cálculo común (Juegos: + dificultad / 2.0 · Tarjetas: + 200 cada 30 días)

El mismo molde: `super.metodo() + lo nuevo`. Ojo con `/ 2.0` (con punto) para que no sea división entera.

```
@Override public double getPuntuacionAjustada() {
    return super.getPuntuacionAjustada() + dificultad / 2.0;
}
@Override public long comision() {
    return super.comision() + 200L * (getDiasCobertura() / 30);
}
```

SI TE PIDEN una solución sin herencia

La consigna acepta composición u otra solución razonable si no duplica la lógica (por ejemplo, un `if (isAsegurado())` adentro de `comision()` en el dominio). Nunca en el colector ni en el main. Elegí una y explicala en `DECISIONES.md`.

5 La operación nueva, en el colector

Dónde: la clase colector (CarteraCreditos, Cloud, Taller, Colector, Servicio).

Casi siempre ya hay un método con `groupingBy` en esa clase: copiá el patrón y cambiá la clave o lo que se hace con cada grupo.

Te piden...	Receta	Devuelve
Conteo por X	`groupingBy(T::getX, Collectors.counting())`	`Map<String, Long>`
Suma por X	`groupingBy(T::getX, Collectors.summingDouble(T::monto))`	`Map<String, Double>`
Total general	`stream().mapToDouble(T::monto).sum()`	`double`
Total de un estado	`filter(t -> t.getEstado().equals("ENTREGADA")).mapToDouble(...).sum()`	`double`

Promedio	`mapToDouble(T::monto).average().orElse(0)`	`double`

Antes · Fintech

```
public class CarteraCreditos {
    ...
    public Map<String, Double> maximaTasaInteresPorTipo() { ... }
}
```

Después · lo nuevo, marcado

```
public class CarteraCreditos {
    ...
    public Map<String, Double> maximaTasaInteresPorTipo() { ... }

    public Map<String, Long> cantidadPorNivelMorosidad() {
        return creditos.stream()
                .collect(Collectors.groupingBy(
                        SolicitudCredito::getNivelMorosidad,    // la clave
                        Collectors.counting()));                // qué hacer con cada grupo
    }
}
```

Colección vacía: `groupingBy` devuelve un mapa vacío y `sum()` devuelve 0. Es justo lo que piden ("resultado vacío o conteos cero").

SI TE PIDEN contar por un dato que el objeto no guarda (Cloud Center: conteo por estado)

El `Servidor` del preenunciado no guardaba el estado. Agregá el atributo y el getter (con el constructor viejo delegando) y recién ahí agrupá.

```
public Map<String, Long> conteoPorEstado() {
    return servidores.stream().collect(Collectors.groupingBy(Servidor::getEstado,
Collectors.counting()));
}
```

SI TE PIDEN una clave que no es un campo (Tarjetas: SIN_SEGURO / ASEGURADO)

La clave se calcula con una lambda. "No se pide agrupar por plazo": leé bien qué clave quieren.

```
groupingBy(c -> c.isAsegurado() ? "ASEGURADO" : "SIN_SEGURO", Collectors.counting())
```

SI TE PIDEN "por tipo" cuando el tipo es la clase (Juegos: Normal / Cooperativo)

Un `getTipo()` en el padre que la subclase redefine, y se agrupa por eso. Sin `instanceof`.

```
public String getTipo() { return "NORMAL"; }                  // en Juego
@Override public String getTipo() { return "COOPERATIVO"; }   // en JuegoCooperativo
groupingBy(Juego::getTipo, Collectors.counting())
```

SI TE PIDEN un total de un estado y un agrupamiento (Taller: ENTREGADA y por dispositivo)

Leé bien el campo: "dispositivo" y "marca" son columnas distintas.

```
reparaciones.stream().filter(r ->
r.getEstado().equals("ENTREGADA")).mapToDouble(Reparacion::importe).sum();
reparaciones.stream().collect(Collectors.groupingBy(Reparacion::getDispositivo,
Collectors.counting()));
```

6 El main: el CSV del parcial y solo delegar

Dónde: App / Main.

Cambiá la ruta por defecto (y seguí aceptando un argumento) e imprimí el resumen, el conteo nuevo y los totales **llamando métodos**. Si en el main aparece un `for`, un `stream` o una cuenta, eso va en otra clase.

Antes · Fintech

```
Path archivo = Path.of(args.length == 0 ? "datos/datos.csv" : args[0]);
ResultadoProceso resultado = new ParserSolicitudes().procesar(archivo);
CarteraCreditos cartera = new CarteraCreditos(resultado.getCreditos());
System.out.println(resultado.informe());
System.out.println(cartera.informe());
int normal = 0, enMora = 0;
for (SolicitudCredito s : resultado.getCreditos()) { ... }   // cuenta en el main
```

Después · lo nuevo, marcado

```
Path archivo = Path.of(args.length == 0 ? "datos/datos-parcial.csv" : args[0]);
ResultadoProceso resultado = new ParserSolicitudes().procesar(archivo);
CarteraCreditos cartera = new CarteraCreditos(resultado.getCreditos());
System.out.println(resultado.informe());
System.out.println(cartera.informe());
Map<String, Long> porMorosidad = cartera.cantidadPorNivelMorosidad();   // delega
System.out.println("NORMAL: %d | EN_MORA: %d".formatted(
        porMorosidad.getOrDefault("NORMAL", 0L),     // 0 si no aparece
        porMorosidad.getOrDefault("EN_MORA", 0L)));
```

SI TE PIDEN mostrar cantidades de algunos valores (Cloud Center: ALERTA / CRITICO)

Lo mismo con `getOrDefault`: es presentación, está bien en el main.

SI TE PIDEN algo que el dominio no tiene ("totales por zona" en Fintech)

La consigna venía de otro parcial. Mostrá lo más parecido que tenga el dominio (en Fintech, la tasa promedio y la máxima por tipo) y aclaralo en `DECISIONES.md`.

SI TE PIDEN un "panorama" o resumen completo (Juegos)

Un método en el servicio que arma el texto (`panorama()`) y el main solo lo imprime.

7 Pruebas y entrega

Dónde: la terminal y la carpeta del proyecto.

1. `mvn test`: todo en verde.
2. Corré el main con el CSV del parcial y revisá que la suma de los contadores dé el total de leídas.
3. Corré también el CSV viejo: tiene que dar exactamente los números del preenunciado.
4. Si un test nuevo contradice la consigna, seguí la consigna y anotalo en `DECISIONES.md`.
5. Escribí `DECISIONES.md` (3 a 5 líneas) y armá el ZIP **sin** `target/`, `.git/` ni archivos del IDE.

Terminal (PowerShell)

```
mvn test
java -cp target/classes ar.edu.backend.App datos/datos-parcial.csv
Compress-Archive -Path pom.xml, src, datos, DECISIONES.md -DestinationPath entrega.zip
```

DECISIONES.md de ejemplo (Fintech)

```
- Un solo parser: la cabecera define el ancho esperado (5 o 6); cada fila se compara contra ese
ancho.
- La morosidad se valida en SolicitudCredito.desdeCampos: RECHAZADO exige EN_MORA; EN_MORA pide
  ingresos > 25000 y deudas <= 12 %. El formato de 5 columnas usa NORMAL por defecto.
- Se conserva el constructor de 4 parámetros (delega con this) para no romper los tests existentes.
- CarteraCreditos.cantidadPorNivelMorosidad() cuenta con groupingBy; el main solo delega.
- "Totales por zona" no aplica a este dominio: se muestran tasa promedio y máxima por tipo de
cliente.
```

Antes de entregar, de un vistazo: dos cabeceras y un solo loop · el dominio acepta los dos anchos y lee la columna nueva solo si existe · atributo + getter y el constructor viejo delega · la regla nueva en el dominio (subclase si cambia un cálculo) · ancho → descarte → validaciones · la operación nueva en el colector (vacío da vacío o 0) · el main con el CSV del parcial, solo delegando · tests en verde, DECISIONES.md y el ZIP sin target/.